

SDEV 2351

DAY 5: FLUXOR — EFFECTS AND ASYNC OPERATIONS

THE MISSING PIECE FOR API CALLS

Session Objectives

1. Explain why effects exist (reducers are pure — effects handle side effects)
2. Implement the triple-action pattern (Load/Success/Failure) for async data loading
3. Create effect methods using `[EffectMethod]` with dependency injection
4. Wire loading and error state into the Fluxor store and component UI

Today's Schedule

PHASE	TIME	WHAT'S HAPPENING
Connecting	10 min	Gap framing — why reducers can't call APIs
Exploring	25 min	Triple-action pattern, effects with DI, product loading demonstration
Practicing	65 min	Todo loading effect, product catalog build, retry logic
Assessing	10 min	Flow-trace exercise + BYSResult reading assignment

PHASE 1 · 10 MINUTES

CONNECTING

Reducers can't call APIs — so what does?

The Gap

Yesterday's reducers are **pure** — same input, same output, no side effects.

- Can a reducer call an API? **No** — it's static, synchronous, pure.
- Can a reducer `await` anything? **No.**
- So where does async work go?

Effects are the answer. An effect listens for an action, does async work, then dispatches a NEW action with the result. The reducer handles that result — still pure.

The Full Flow

Happy path:

Component dispatches `LoadProductsAction` → Effect calls `IProductService.GetAllAsync()` → Effect dispatches `LoadProductsSuccessAction(products)` → Reducer updates state → Component re-renders

Failure path:

Same trigger → Effect calls service → Exception → Effect dispatches `LoadProductsFailureAction(error)` → Reducer sets ErrorMessage → Component re-renders with error

How many actions? Three: one trigger, one success, one failure. The branch happens inside the effect's try/catch.

PHASE 2 · 25 MINUTES

EXPLORING

Triple-action pattern, [EffectMethod], and dependency injection in effects

The Triple-Action Pattern

Every async data operation follows this template:

ACTION	DISPATCHED BY	PURPOSE
<code>LoadProductsAction</code>	Component	Trigger — starts the operation
<code>LoadProductsSuccessAction(List<Product>)</code>	Effect	Carries loaded data
<code>LoadProductsFailureAction(string Error)</code>	Effect	Carries error message

Each action has a corresponding reducer that updates state.

Effects vs Reducers

REDUCERS (DAY 4)

Static methods

Synchronous only

No DI, no side effects

Take state + action → return new state

[ReducerMethod]

EFFECTS (DAY 5)

Instance methods

Async/await allowed

Constructor DI for services

Receive action + IDispatcher

[EffectMethod]

Effects are WHERE async happens. Reducers stay pure. This separation is the key to testable state management.

State with Loading and Error Properties

```
[FeatureState]
public record ProductState
{
    public List<Product> Items { get; init; } = [];
    public bool IsLoading { get; init; }
    public string? ErrorMessage { get; init; }
}
```

- **Trigger reducer:** sets `IsLoading = true`
- **Success reducer:** sets `Items`, clears `IsLoading`
- **Failure reducer:** sets `ErrorMessage`, clears `IsLoading`

Full code: [see reference-sheet.md](#) or your IDE

Effect with Dependency Injection

```
public class ProductEffects(IProductService svc)
{
    [EffectMethod]
    public async Task Handle(
        LoadProductsAction action, IDispatcher dispatcher)
    {
        try { var items = await svc.GetAllAsync();
            dispatcher.Dispatch(new LoadProductsSuccessAction(items)); }
        catch (Exception ex) {
            dispatcher.Dispatch(new LoadProductsFailureAction(ex.Message)); }
    }
}
```

Fluxor resolves `IProductService` automatically through the DI container.

Full code: [see reference-sheet.md](#) or your IDE

Demonstration: Product Loading Effect

Built live in `Day05Demo` — **6 files**: three already written and narrated, three typed on screen.

STEP	FILE	STATUS
1	Three actions — Trigger, Success, Failure	Already written — narrated
2	<code>ProductState</code> — adds <code>IsLoading</code> + <code>ErrorMessage</code>	Already written — narrated
3	<code>ProductReducers</code> — all three actions	Typed live
4	<code>ProductEffects</code> — DI + <code>[EffectMethod]</code> try/catch	Typed live
5	<code>ProductFeature</code> — the Day 4 <code>Feature<T></code> pattern	Already written — narrated
6	<code>Products.razor</code> — dispatch in <code>OnInitialized</code>	Typed live

Then: run for the loading spinner (1-second delay), and again with the mock throwing for the error state.

The async flow is **component** → **effect** → **service** → **success/failure action** → **reducer** → **UI**. You rebuild this same pattern yourself in Practice 2.

PHASE 3 · 65 MINUTES

PRACTICING

Build effects for todo loading, product catalog, and retry logic

Pre-Lab 1 Environment Check

UNGRADED · SETUP VERIFICATION

Complete the setup checks **alongside today's practices** (self-paced):

- Clone the repository, confirm it builds, configure your git identity
- Confirms your machine is ready before MA #1 (due Day 9) and Lab 1 (starts Day 9)
- Not graded, and **not** the assessment -- the Git workflow itself is MA #1, next slide

Work on this between practices or when you finish a practice early.

Micro-Assessment #1 — Assigned Today

GRADED · 2% · CO6 · DUE DAY 9

Git Branching Workflow. The check above confirms your setup — this is the actual work. In **your own GitHub Classroom repo**:

- Branch, make 2–3 conventional commits, push, open a PR
- **Create and resolve a merge conflict** — your repo ships with the conflict site ready
- Merge, then delete the branch
- Commit a short proof-of-self file to that repo

Paste your repo URL in Brightspace — that is all that goes there. The work lives in your Classroom repo.

It's **open-book**: keep Git Branching — Your Complete Lab Guide open while you work. Section 10 walks the merge conflict end to end, and your repo ships with the conflict site already set up. Section 14 is your pre-submit self-check.

Practice 1: Todo Loading Effect (worked launch, 15 min)

Practice starter provides: Day 4's todo store working, `ITodoService` registered, and — already done for you — the `IsLoading` / `ErrorMessage` state properties plus the loading and error display markup.

You add:

- `LoadTodosAction`, `LoadTodosSuccessAction`, `LoadTodosFailureAction` in `TodoActions.cs`
- Three `[ReducerMethod]` load reducers in `TodoReducers.cs`, setting those pre-built properties
- `TodoEffects.cs` — constructor DI for `ITodoService`, `[EffectMethod]` with try/catch
- Dispatch `LoadTodosAction` in component `OnInitialized`

The state properties and Razor conditionals are pre-built on purpose — today's target is the effect logic in `.cs`, not the markup.

Practice 2: Product Catalog (Independent, 25 min)

The same triple-action pattern the demonstration just showed. Practice starter provides: `IProductService` + mock, empty `ProductState.cs`, `Products.razor` with loading/error/data UI

You write:

- All three actions (Load, Success, Failure)
- All three reducers
- `ProductEffects` class with constructor DI
- `ProductFeature` class
- Component wiring + a **Refresh** button that re-dispatches `LoadProductsAction`

Practice 3: Retry Logic (Challenge, 10 min)

Extend Practice 2 with error recovery:

- When in error state, display a **Retry** button
- On click: clear the error and re-dispatch `LoadProductsAction`
- Handle empty state: if products load successfully but the list is empty, show "No products found"

Troubleshooting Tips

SYMPTOM	LIKELY CAUSE
Effect never fires	Effect method not <code>async Task</code>
Effect runs but state doesn't update	Missing <code>ProductFeature</code> class
"Cannot resolve service" error	Service not registered in <code>Program.cs</code> — a <u>demo-project</u> concern only; registration is pre-built in your practice starter
Success action dispatched but UI unchanged	Dispatching in reducer instead of effect
Loading spinner never disappears	Reducer not clearing <code>IsLoading = false</code>

PHASE 4 · 10 MINUTES

ASSESSING

Summary

Today you learned to:

1. **Explain why effects exist** — reducers are pure; effects handle side effects
2. **Implement the triple-action pattern** — Load/Success/Failure for every async operation
3. **Create effects with** `[EffectMethod]` — instance methods with constructor DI and async/await
4. **Wire loading and error state** — `IsLoading` and `ErrorMessage` properties in the store

Exit Ticket

In one sentence: why can't a Fluxor reducer call an API? What handles async work instead?

Flow-trace fill-in-the-blanks:

1. User clicks "Load Products". The component dispatches ____
2. The ____ catches this action and calls `IProductService`
3. The service returns data. The effect dispatches ____
4. The ____ handles this action and returns new state
5. The component re-renders because it inherits from ____

What's Next

Day 6: Centralized State, Component Integration, and the BYSResult Pattern

- When should state live in the store vs. stay local in a component?
- Multiple components subscribing to the same store slice
- The **BYSResult** `Result<T>` pattern — cleaner than try/catch for service errors

Pre-class reading assigned: BYSResult Pattern Guide (~20 min). Linked from the Day 6 page in Brightspace. Focus on Core Concepts (sections 1-4), Basic Pattern (section 5), Integration with Fluxor (section 9).